



INFORME DEL TRABAJO

I. PORTADA

Tema:	TDA Laboratorio-Gestor de impresiones
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumnos participantes:	Tisalema Guashco Darwin Joel
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DEL TRABAJO

2.1 Objetivos

General:

Desarrollar un programa en Java que gestione trabajos de impresión mediante los tipos de datos abstractos cola y pila, permitiendo registrar, procesar y consultar los documentos enviados a impresión.

Específicos:

- Implementar una cola (FIFO) para almacenar los documentos pendientes y garantizar que se impriman en el mismo orden en que fueron enviados
- Implementar una pila (LIFO) que registre el historial de impresiones, de modo que el último documento impreso sea el primero en consultarse o deshacerse.
- Verificar el funcionamiento del programa mediante una prueba de escritorio y diagramas que muestren el estado de la cola y la pila en cada operación.

2.2 Ejercicio

1. Ejercicio

Desarrollar en Java un programa que administre una cola de impresión usando:

una estructura dinámica de tipo Deque (ArrayDeque);

una cola (FIFO) para los documentos pendientes;

una pila (LIFO) para el historial de impresiones;

al menos dos tipos de datos (String y boolean implícito en las validaciones);

una clase que encapsule la estructura (TDA);

instanciación de objetos;

métodos públicos y privados (encapsulamiento);

comentarios que identifiquen cada concepto.

2. Solución propuesta

Se crea el TDA GestorImpresiones, que administra internamente dos estructuras: una cola de documentos pendientes y una pila con el historial de documentos ya impresos. La clase Main instancia el gestor y ejecuta la secuencia de pruebas.

Clase	Responsabilidad
-------	-----------------



GestorImpresiones	TDA que administra la cola de pendientes y la pila de historial.
Main	Instancia el TDA y ejecuta las operaciones de prueba.
ArrayDeque<String> (java.util)	Implementación concreta de Deque usada como cola y como pila.

3. ¿Qué es un TDA?

Un Tipo de Dato Abstracto define: 1) los datos que puede almacenar; 2) las operaciones que pueden realizarse; 3) las reglas (invariantes) que siempre deben cumplirse.

En este ejercicio el TDA es GestorImpresiones:

```
public class GestorImpresiones {  
    private Deque<String> pendientes; // COLA (FIFO)  
    private Deque<String> historial; // PILA (LIFO)  
}
```

Los datos están encapsulados porque pendientes e historial son privados. Para trabajar con ellos se usan métodos públicos:

```
gestor.registrarDocumento("informe.pdf");  
gestor.imprimirSiguiente();  
gestor.recuperarUltima();  
gestor.mostrarEstado();
```

4. Relación entre requisitos y código

Requisito	Aplicación
Estructura de datos	Deque<String> implementado con ArrayDeque.
Cola (FIFO)	pendientes: offerLast() al final, pollFirst() al frente.
Pila (LIFO)	historial: push() y pop() sobre el mismo extremo.
Clase	GestorImpresiones y Main.
Instanciación	new ArrayDeque<>() y new GestorImpresiones().
Encapsulamiento	Atributos private y método privado guardarImpresion().
Validación de estado	isEmpty() antes de extraer elementos.
Comentarios	El código incluye marcas CONCEPTO:..

5. Paso 1: Importar las clases de java.util

```
import java.util.ArrayDeque;  
import java.util.Deque;
```

Deque es la interfaz (doble cola) y ArrayDeque es la implementación concreta. Programar contra la interfaz permite cambiar la implementación sin modificar el resto del código.



6. Paso2: Declarar los atributos del TDA

```
private Deque<String> pendientes; // se usará como COLA  
private Deque<String> historial; // se usará como PILA
```

Ambos atributos son privados: ninguna clase externa puede modificar las estructuras directamente. String almacena el nombre del documento y no es un tipo primitivo.

7. Paso3: Construir el objeto (instanciación)

```
public GestorImpresiones() {  
    pendientes = new ArrayDeque<>();  
    historial = new ArrayDeque<>();  
}
```

El constructor crea las dos estructuras vacías. Sin esta instanciación los atributos serían null y cualquier operación lanzaría NullPointerException.

8. Paso4: Registrar documentos (encolar, FIFO)

```
public void registrarDocumento(String nombre) {  
    pendientes.offerLast(nombre); // entra por el FINAL de la cola  
    System.out.println("Registrado: " + nombre);  
}
```

offerLast() agrega el documento al final. El primero que entra será el primero en imprimirse: comportamiento FIFO.

9. Paso5: Imprimir el siguiente (desencolar)

```
public void imprimirSiguiente() {  
    if (pendientes.isEmpty()) { // validación de cola vacía  
        System.out.println("No hay documentos pendientes para imprimir");  
        return;  
    }  
    String nombre = pendientes.pollFirst(); // sale por el FRENTE  
    System.out.println("Imprimiendo: " + nombre);  
    guardarImpresion(nombre);  
}
```

pollFirst() extrae el elemento del frente. Antes se comprueba isEmpty() para no operar sobre una cola vacía; pollFirst() devolvería null y el programa quedaría inconsistente.

10. Paso6: Guardar en el historial (apilar, LIFO)

```
private void guardarImpresion(String nombre) {  
    historial.push(nombre); // entra por el TOPE de la pila  
}
```

El método es private: solo el propio TDA puede escribir en el historial. push() inserta en el tope, por lo que el último documento impreso queda siempre accesible primero.



11. Paso7: Recuperar la última impresión (desapilar)

```
public void recuperarUltima() {  
    if (historial.isEmpty()) {  
        System.out.println("No hay impresiones en el historial para recuperar");  
        return;  
    }  
    String nombre = historial.pop();    // sale del TOPE de la pila  
    pendientes.addFirst(nombre);        // vuelve al FRENTE de la cola  
    System.out.println("Recuperado al frente: " + nombre);  
}
```

Esta operación combina las dos estructuras: pop() saca el último impreso y addFirst() lo coloca al frente de la cola para reimprimirlo de inmediato.

12. Estructuras y tipos de datos utilizados

Elemento	Categoría	Uso
String	Referencia (no primitivo)	nombre del documento.
Deque<String>	Interfaz genérica	tipo declarado de pendientes e historial.
ArrayDeque<String>	Clase (referencia)	implementación real de ambas estructuras.
boolean	Primitivo	valor devuelto por isEmpty() en las validaciones.
void	Tipo de retorno	todos los métodos de operación.

Operación	Cola (pendientes)	Pila (historial)
Insertar	offerLast() / addFirst()	push()
Extraer	pollFirst()	pop()
Consultar vacío	isEmpty()	isEmpty()

13. Diagrama de clases

Diagrama de clases - Gestor de Impresiones

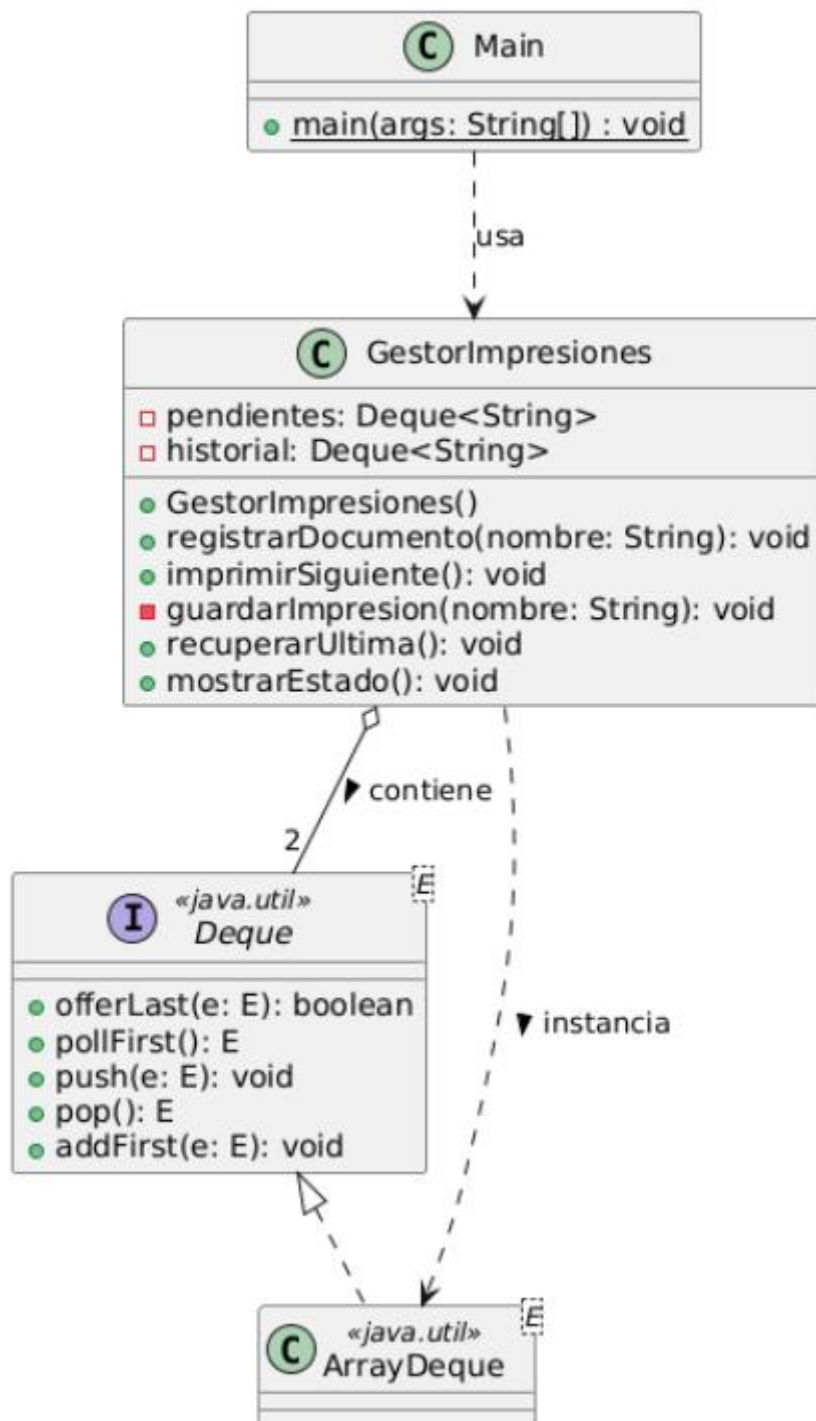


Figura1. Diagrama de clases

14. Especificación del TDA

Estado

pendientes : Deque<String> - documentos en espera, orden FIFO

historial : Deque<String> - documentos impresos, orden LIFO



Operaciones

Método	Resultado
GestorImpresiones()	Crea las dos estructuras vacías.
registrarDocumento(String)	Agrega un documento al final de la cola.
imprimirSiguiente()	Extrae el documento del frente y lo envía al historial; avisa si la cola está vacía.
guardarImpresion(String)	(privado) Apila el documento impreso.
recuperarUltima()	Desapila la última impresión y la coloca al frente de la cola; avisa si el historial está vacío.
mostrarEstado()	Muestra el contenido de la cola y de la pila.

Invariantes

1. pendientes e historial nunca son null después del constructor.
2. Un documento está en la cola o en el historial, no en ambos a la vez.
3. imprimirSiguiente() solo extrae si pendientes no está vacía.
4. recuperarUltima() solo extrae si historial no está vacío.
5. El primer documento registrado es el primero en imprimirse (FIFO).
6. El último documento impreso es el primero en recuperarse (LIFO).
7. El tamaño total de documentos nunca es negativo.

15. Pseudocódigo

INICIO

instanciar GestorImpresiones (cola vacía, pila vacía)

imprimirSiguiente -> cola vacía: mostrar aviso

recuperarUltima -> pila vacía: mostrar aviso

mostrarEstado

registrar "informe.pdf"

registrar "tarea.docx"

registrar "foto.png"

mostrarEstado

imprimirSiguiente -> sale informe.pdf, se apila

imprimirSiguiente -> sale tarea.docx, se apila

mostrarEstado

recuperarUltima -> tarea.docx vuelve al frente

mostrarEstado

imprimirSiguiente -> tarea.docx



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



imprimirSiguiente -> foto.png
imprimirSiguiente -> cola vacía: mostrar aviso
mostrarEstado

recuperarUltima -> foto.png vuelve al frente
mostrarEstado

FIN

16. Prueba de escritorio

Paso	Operación	Pendientes (frente → final)	Historial (tope → base)	Resultado
1	Crear el gestor	[]	[]	Ambas estructuras vacías.
2	imprimirSiguiente()	[]	[]	Aviso: no hay documentos pendientes.
3	recuperarUltima()	[]	[]	Aviso: historial vacío.
4	registrar informe.pdf	[informe.pdf]	[]	Encolado al final.
5	registrar tarea.docx	[informe.pdf, tarea.docx]	[]	Encolado al final.
6	registrar foto.png	[informe.pdf, tarea.docx, foto.png]	[]	Encolado al final.
7	imprimirSiguiente()	[tarea.docx, foto.png]	[informe.pdf]	Imprime informe.pdf y lo apila.
8	imprimirSiguiente()	[foto.png]	[tarea.docx, informe.pdf]	Imprime tarea.docx y lo apila.
9	recuperarUltima()	[tarea.docx, foto.png]	[informe.pdf]	tarea.docx vuelve al frente.
10	imprimirSiguiente()	[foto.png]	[tarea.docx, informe.pdf]	Imprime tarea.docx.
11	imprimirSiguiente()	[]	[foto.png, tarea.docx, informe.pdf]	Imprime foto.png.
12	imprimirSiguiente()	[]	[foto.png, tarea.docx, informe.pdf]	Aviso: no hay pendientes.
13	recuperarUltima()	[foto.png]	[tarea.docx, informe.pdf]	foto.png vuelve al frente.

2.3 Análisis de resultados

Implementación del TDA



El TDA GestorImpresiones cumple con la totalidad de los requisitos: uso de los TDAs cola (FIFO) para los trabajos pendientes y pila (LIFO) para el historial, operaciones completas de registro, impresión y consulta, control de la cola vacía antes de imprimir, y encapsulamiento total de los atributos mediante una clase en Java.

Gestión de los datos

Veredicto: la combinación cola + pila es la estructura más adecuada para el problema.

Justificación:

- La cola (ArrayDeque) garantiza que los documentos se impriman en el mismo orden en que fueron enviados, sin importar cuántos se registren.
- La pila permite recuperar el historial en orden inverso, de modo que el último documento impreso sea el primero en consultarse.
- Al usar la clase ArrayDeque de Java, no se requieren índices ni desplazamientos manuales de elementos: las operaciones offer/poll y push/pop se ejecutan de forma directa y eficiente

2.4 Conclusiones

- El TDA GestorImpresiones logró encapsular correctamente las dos estructuras de datos (pendientes e historial) y las reglas de impresión, manteniendo el orden de llegada de los documentos
- La cola resultó indispensable para garantizar el principio FIFO, mientras que la pila permitió implementar el historial con principio LIFO, mostrando que cada estructura responde a una necesidad distinta dentro del mismo problema.
- El uso de ArrayDeque simplificó la implementación al proveer ambas operaciones en una sola clase, evitando errores comunes de manejo manual de arreglos.
- La prueba de escritorio y los diagramas verificaron el estado de la cola y la pila en cada operación, confirmando que los documentos se imprimen en orden y que el historial refleja correctamente la secuencia inversa de impresiones

2.5 Referencias Bibliográficas

- [1] Baeldung, "Reduce Object Header Size and Save Memory in Java 25," *Baeldung*, may 2026. [En línea]. Disponible: <https://www.baeldung.com/java-object-header-reduced-size-save-memory>. [Accedido: 26-ago-2026].
- [2] "The Cost of Garbage Collection for State Machine Replication," *arXiv preprint*, arXiv:2405.11182, 2024. [En línea]. Disponible: <https://arxiv.org/pdf/2405.11182>. [Accedido: 26-ago-2026].

2.6 Anexos

Código comentado

GestorImpresiones.java

```
package laboratorio;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
import java.util.ArrayDeque; // CONCEPTO: implementación concreta
import java.util.Deque;      // CONCEPTO: interfaz (cola doble)

// CONCEPTO: CLASE / TDA que encapsula una cola y una pila
public class GestorImpresiones {

    // CONCEPTO: ENCAPSULAMIENTO (atributos privados)
    private Deque<String> pendientes; // CONCEPTO: COLA (FIFO)
    private Deque<String> historial;  // CONCEPTO: PILA (LIFO)

    // CONCEPTO: CONSTRUCTOR e INSTANCIACIÓN de objetos
    public GestorImpresiones() {
        pendientes = new ArrayDeque<>();
        historial = new ArrayDeque<>();
    }

    // CONCEPTO: ENCOLAR al final (FIFO)
    public void registrarDocumento(String nombre) {
        pendientes.offerLast(nombre);
        System.out.println("Registrado: " + nombre);
    }

    // CONCEPTO: DESENCOLAR del frente + validación de cola vacía
    public void imprimirSiguiente() {
        if (pendientes.isEmpty()) {
            System.out.println("No hay documentos pendientes para imprimir");
            return;
        }
        String nombre = pendientes.pollFirst();
        System.out.println("Imprimiendo: " + nombre);
        guardarImpresion(nombre);
    }

    // CONCEPTO: MÉTODO PRIVADO (solo lo usa la propia clase) - APILAR
    private void guardarImpresion(String nombre) {
        historial.push(nombre);
    }

    // CONCEPTO: DESAPILAR (LIFO) y devolver al frente de la cola
    public void recuperarUltima() {
        if (historial.isEmpty()) {
            System.out.println("No hay impresiones en el historial para recuperar");
            return;
        }
        String nombre = historial.pop();
        pendientes.addFirst(nombre);
        System.out.println("Recuperado al frente: " + nombre);
    }
}
```



}

// CONCEPTO: OPERACIÓN DE CONSULTA del estado del TDA

```
public void mostrarEstado() {  
    System.out.println("Pendientes: " + pendientes);  
    System.out.println("Historial : " + historial);  
    System.out.println();  
}  
}
```

Main.java

package laboratorio;

```
public class Main {  
    public static void main(String[] args) {  
        // CONCEPTO: INSTANCIACIÓN del TDA  
        GestorImpresiones gestor = new GestorImpresiones();  
  
        gestor.imprimirSiguiente(); // caso borde: cola vacía  
        gestor.recuperarUltima();   // caso borde: pila vacía  
        gestor.mostrarEstado();  
  
        gestor.registrarDocumento("informe.pdf");  
        gestor.registrarDocumento("tarea.docx");  
        gestor.registrarDocumento("foto.png");  
        gestor.mostrarEstado();  
  
        gestor.imprimirSiguiente();  
        gestor.imprimirSiguiente();  
        gestor.mostrarEstado();  
  
        gestor.recuperarUltima();  
        gestor.mostrarEstado();  
  
        gestor.imprimirSiguiente();  
        gestor.imprimirSiguiente();  
        gestor.imprimirSiguiente();  
        gestor.mostrarEstado();  
  
        gestor.recuperarUltima();  
        gestor.mostrarEstado();  
    }  
}
```



```
No hay documentos pendientes para imprimir
No hay impresiones en el historial para recuperar
Pendientes: []
Historial : []

Registrado: informe.pdf
Registrado: tarea.docx
Registrado: foto.png
Pendientes: [informe.pdf, tarea.docx, foto.png]
Historial : []

Imprimiendo: informe.pdf
Imprimiendo: tarea.docx
Pendientes: [foto.png]
Historial : [tarea.docx, informe.pdf]

Recuperado al frente: tarea.docx
Pendientes: [tarea.docx, foto.png]
Historial : [informe.pdf]

Imprimiendo: tarea.docx
Imprimiendo: foto.png
No hay documentos pendientes para imprimir
Pendientes: []
Historial : [foto.png, tarea.docx, informe.pdf]

Recuperado al frente: foto.png
Pendientes: [foto.png]
Historial : [tarea.docx, informe.pdf]
```

Figura 2. Ejecución del programa en Java.

A continuación, se debe adjuntar el enlace del repositorio donde se podrá visualizar los códigos realizados en Java, además de las fotografías de ejecución junto con el diagrama UML y el presente documento en formato .docx y .pdf.

<https://github.com/Joel03032007/Laboratorio>